

Viva Questions & Answers

Restaurant Table Reservation & Waitlist System

Java Programming - Case Study 70 | Aditya Sunil Chouksey (150096725070)

One-minute project summary (learn this by heart)

"My project is a Restaurant Table Reservation and Waitlist System in Java. A restaurant host uses it to manage bookings, walk-in guests and seating across tables of different sizes and sections.

Tables, reservations, customers and the waitlist are classes, and table status is an enum - Available, Reserved, Occupied. Reservations are stored in an ArrayList, looked up by ID with a HashMap, and kept in time order in a TreeMap. Walk-ins wait in a LinkedList queue, because guests join at the end and are served first-come-first-served.

New bookings get the smallest table that fits, and each party blocks its table for 90 minutes, so double bookings are impossible. When guests leave, a background thread waits 2 seconds for the table to be reset, then offers it to the booking due next or to the first waiting party that fits, and alerts the host. A second thread holds tables 30 minutes before booked guests arrive.

All input is validated with custom exceptions, data is saved automatically, and everything is operated through a Swing GUI with four tabs."

82 questions in 10 groups. Answers are written to be said aloud - short, plain and specific to this project.

A. The project in general

Q1. What does your project do?

It is a Java desktop application for a restaurant host. It manages table reservations and the walk-in waitlist, assigns each party the smallest table that fits, and alerts the host when a table becomes free for a booked guest or for the next waiting party.

Q2. Who is the user?

The host at the front desk. Guests never use the system directly.

Q3. What are the seven modules?

Table Setup, Reservation Booking, Waitlist Management, Table Assignment, Availability Notification, Reservation Search and Seating Reports. Each has a tab or a service method behind it.

Q4. How is the project organised?

Six packages: model (data), service (all rules and threads), persistence (saving), exception, util (validation and dates) and gui (Swing). The GUI never makes decisions - it calls RestaurantManager.

Q5. Which class is the heart of the system?

RestaurantManager. It owns the tables, reservations and waitlist, and every operation - booking, seating, releasing, searching - goes through it.

Q6. How many classes and lines of code?

33 source files, about 3,400 lines, using only the standard JDK.

Q7. What happens when the application starts?

Main checks there is a screen, loads data/restaurant.dat (or sample data on the first run), opens the window on the Event Dispatch Thread and starts the ReservationMonitor thread.

Q8. What was the hardest part?

Making the waitlist fair and safe at the same time: a free table must go to the right party, must not be taken by someone who arrived later, and must not be given to a walk-in when a booked party needs it soon - all while two background threads and the GUI share the same data.

B. Classes, objects, constructors and enums**Q9. Name the main model classes.**

Table, Reservation, Customer, WaitlistEntry and Waitlist, plus the enums TableStatus, ReservationStatus and Section.

Q10. What does the Table constructor guarantee?

Every new table starts AVAILABLE with nobody holding or occupying it.

Q11. How does a reservation get its ID?

The constructor calls a static AtomicInteger counter: bookings get "R" plus the number (R1001), seated walk-ins "WI" (WI1002).

Q12. Why AtomicInteger?

Two threads could create reservations at the same moment; incrementAndGet() is atomic, so they can never receive the same number.

Q13. What are the TableStatus values?

AVAILABLE, RESERVED and OCCUPIED, exactly as the brief specifies.

Q14. Why an enum instead of strings?

The compiler only accepts the defined values, so a typo cannot create an invalid status; enums also work in switch statements and as EnumMap keys.

Q15. What does ReservationStatus.isActive() do?

Returns true for CONFIRMED and SEATED - the statuses that still block a table. Completed, cancelled and no-show bookings do not.

Q16. What is constructor chaining in your code?

RestaurantManager() calls this(Clock.systemDefaultZone()), so both constructors share one initialisation path.

Q17. Where do you use encapsulation?

All fields are private; state changes go through methods like table.hold(), and getTables() returns a copy so outside code cannot modify the manager's list.

C. Collections**Q18. Which collections do you use and for what?**

ArrayList for tables and reservations, HashMap for reservation by ID, TreeMap for reservations by time, and LinkedList for the waitlist queue.

Q19. Why a HashMap?

Looking up a reservation by ID is $O(1)$ on average instead of scanning the whole list - used for check-in, edit, cancel and search by ID.

Q20. Why a TreeMap?

It keeps keys sorted. With start time as the key, subMap() gives "bookings due in the next 30 minutes" and tailMap() gives the next booking for a table directly.

Q21. Why is the TreeMap value a List<Reservation>?

Two bookings can start at the same minute; a key can appear only once, so each key maps to a list.

Q22. Why a LinkedList for the waitlist?

It is a queue: parties join at the end and leave from the front or the middle. LinkedList does those in $O(1)$ with an iterator; an ArrayList would shift elements.

Q23. How do you remove a party from the middle of the waitlist?

With an Iterator and it.remove(); removing inside a for-each loop would throw ConcurrentModificationException.

Q24. How do you keep the three reservation structures consistent?

Every change goes through index(), unindexTime() and the delete method, which update the ArrayList, HashMap and TreeMap together inside one synchronized method.

Q25. What is the time complexity of a TreeMap get?

$O(\log n)$ - it is a balanced (red-black) tree.

Q26. Does a HashMap keep insertion order?

No. That is exactly why a separate TreeMap is used when order matters.

Q27. What is the difference between ArrayList and LinkedList?

ArrayList is a resizable array - fast index access, slow insert/remove in the middle. LinkedList is a chain of nodes - fast insert/remove at known positions, slow index access.

D. CRUD, searching and sorting**Q28. Show CRUD in your project.**

Create: createReservation. Read: getReservation / searchReservations. Update: updateReservation plus checkIn, cancel and no-show. Delete: deleteReservation.

Q29. What is the difference between Cancel and Delete?

Cancel keeps the record with status CANCELLED so reports still count it; Delete removes it from every structure. A seated party cannot be deleted until its table is released.

Q30. How does search work?

By ID it uses the HashMap. By table it compares table numbers. By customer it does a case-insensitive contains() on the name and a digits-only contains() on the phone.

Q31. Is your search linear or binary?

Linear (plus the HashMap for IDs), because "contains" matching cannot use binary search and the data is small.

Q32. How is sorting implemented?

The SortField enum stores a Comparator for each order; sort() copies the list and calls Collections.sort with that comparator or comparator.reversed().

Q33. What does thenComparing do?

Breaks ties: parties of the same size are then ordered by time.

Q34. Comparable or Comparator - which did you use and why?

Comparator, because five different orders are needed; Comparable allows only one natural order.

Q35. What sorting algorithm does Collections.sort use?

TimSort - $O(n \log n)$, and stable, so equal elements keep their relative order.

E. Table assignment and business rules**Q36. How is a table chosen for a booking?**

Best fit: among tables big enough and free for the whole dining window, the one with the fewest seats; the preferred section is tried first, then any section.

Q37. Why best fit?

It keeps large tables free for large parties. A couple at a six-seat table could force a family of six to wait.

Q38. How do you prevent double booking?

Each party blocks its table for [start, start + 90 min). A new booking conflicts if $\text{newStart} < \text{busyEnd}$ and $\text{newEnd} > \text{busyStart}$.

Q39. Can two bookings be back to back?

Yes. A booking ending at 20:30 and one starting at 20:30 do not overlap because $20:30 < 20:30$ is false.

Q40. When is a table held for a booked guest?

30 minutes before their time: the table turns RESERVED and the host is alerted.

Q41. What if a booked guest is late?

After 15 minutes the host gets one warning to call them or mark a no-show; a no-show frees the table for the next party.

Q42. Why is a walk-in sometimes refused a table that looks free?

Because a booking needs that table within the next 90 minutes - the walk-in would not finish in time.

Q43. How is the waitlist fair?

When a table frees up, the queue is checked from the front and the first party that fits gets it. A new walk-in is only seated directly if no earlier waiting party fits a free table.

Q44. What are the validation limits?

Party size 1-20, table capacity 1-20, service hours 11:00-22:30, 15-minute slots, at most 60 days ahead, names 2-50 letters, phones 10-13 digits.

F. Threads and concurrency**Q45. Where do you use Runnable and Thread?**

WaitlistNotifier (one new thread per released table: sleeps 2 s, then offers the table) and ReservationMonitor (a daemon thread that sweeps every 15 s).

Q46. Why sleep for two seconds?

It simulates the time staff need to clear and reset the table before the next party is called.

Q47. What is the difference between start() and run()?

start() creates a new thread which then calls run(). Calling run() directly runs the code on the current thread - the GUI would freeze.

Q48. What is a daemon thread?

A background thread that does not keep the JVM alive; the monitor stops automatically when the window closes.

Q49. Why are the RestaurantManager methods synchronized?

The GUI, the monitor and notifier threads all change the same tables and lists. synchronized lets only one thread in at a time, preventing race conditions such as a table being seated and held at once.

Q50. What is a race condition?

A bug where the result depends on which thread runs first - for example two threads both seeing a table as free and both assigning it.

Q51. How do background threads update the GUI safely?

They never touch Swing. They call the listener, and MainFrame wraps the update in SwingUtilities.invokeLater so it runs on the Event Dispatch Thread.

Q52. What is the Event Dispatch Thread?

The single thread on which Swing handles all events and painting; Swing components are not thread-safe and must only be used from it.

Q53. Why is the stop flag volatile?

So a change made by one thread is immediately visible to the monitor thread; without it the thread might keep a stale cached value.

Q54. Could your program deadlock?

No: there is only one lock (the manager), background threads sleep outside it, and listeners only queue work with invokeLater instead of waiting for the GUI.

G. Swing GUI

Q55. Describe the GUI.

One JFrame with a header (counts, clock, save status), a yellow alert banner, four tabs - Floor & Tables, Reservations, Walk-ins & Waitlist, Seating Reports - and a notification log at the bottom.

Q56. How do the JTables get their data?

Each panel has a DefaultTableModel; refresh() clears it and adds one row per table, reservation or waiting party. The JTable just draws the model.

Q57. How are status cells coloured?

StatusCellRenderer extends DefaultTableCellRenderer and sets the background by status: green available, amber reserved, red occupied, blue confirmed, grey closed.

Q58. Which layout managers did you use?

BorderLayout for the main areas, FlowLayout for button rows, GridBagLayout for forms and GridLayout for the two-row toolbar.

Q59. How are button clicks handled?

Each button gets an ActionListener written as a lambda, e.g. e -> book(); it calls the manager and shows any exception message in a dialog.

Q60. Why javax.swing.Timer?

Its actions run on the Event Dispatch Thread, so it can update components directly
- used for the clock, the 30-second refresh and the autosave delay.

Q61. Why can the party-size spinner go above 20?

It goes to 99 on purpose, so that the Validator - not the spinner snapping back silently - rejects 21 with a clear message.

H. Exceptions, validation and persistence**Q62. Describe your exception hierarchy.**

RestaurantException extends Exception; InvalidReservationException, InvalidTableException, TableNotAvailableException and NotFoundException extend it.

Q63. Checked or unchecked - why?

Checked, so the compiler forces every caller (each GUI action) to handle business-rule failures.

Q64. Walk me through what happens when someone books a party of 21.

The panel calls createReservation; Validator.validatePartySize throws InvalidReservationException; it propagates up; the panel's catch block shows "Party size cannot exceed 20..." and nothing is saved.

Q65. How do you reject 30 February?

The DateTimeFormatter uses ResolverStyle.STRICT with the pattern "uuuu-MM-dd HH:mm", so impossible dates throw DateTimeParseException, which TimeUtil turns into a friendly error.

Q66. How is data saved?

With Java serialization: one RestaurantSnapshot object is written to a temporary file and moved over data/restaurant.dat; it happens 1.5 s after each change and when the window closes.

Q67. Why write to a temporary file first?

So a crash during saving cannot corrupt the real file - the move replaces it in a single step.

Q68. Why are the HashMap and TreeMap not saved?

They are rebuilt from the reservation list when loading, so they can never be out of step with it.

Q69. What happens if the data file is damaged?

It is renamed restaurant.dat.corrupt-<time> and the app starts with sample data instead of crashing.

I. Design choices - "why did you choose X?"

Q70. Why Swing and not JavaFX?

The brief asks for Swing, and Swing is part of every JDK 8+ with no extra download.

Q71. Why not a database?

The brief is about collections; a single front desk only needs a local file. A database (via JDBC) is listed as future scope for several host computers.

Q72. Why separate the service layer from the GUI?

The rules can be tested and reused without the window, and the GUI stays simple. A web interface could reuse RestaurantManager unchanged.

Q73. Why the Observer pattern?

The manager must tell the GUI about changes made by background threads, without depending on Swing classes.

Q74. Why a 90-minute dining window?

It is a typical table-turn time for a casual dinner and keeps the conflict rule simple; it is one constant that could be made configurable.

Q75. Why is the sample data loaded on the first run?

So the system can be demonstrated immediately with a realistic busy floor - seated walk-ins, bookings and a waitlist.

J. Examiner cross-questions

Q76. Show me where the LinkedList is.

model/Waitlist.java: private final LinkedList<WaitlistEntry> queue; enqueue() calls addLast, firstFitFor() iterates from the front.

Q77. What happens if two hosts click at the same time?

Both actions call synchronized manager methods, so one completes before the other starts; the second sees the updated state.

Q78. What if every table is occupied and a booking is due?

The booking keeps its assigned table; when that table is released, processAvailability holds it for the booking first, before the waitlist.

Q79. Can a party of 12 be seated?

Yes, at the 12-seat private room table if it is free; otherwise they wait or are refused with "No table for a party of 12 is free...".

Q80. What if the host deletes a table that has a booking?

removeTable throws InvalidTableException naming the booking; the host must move or cancel it first.

Q81. How would you scale this to multiple branches?

Move storage to a database, add a restaurant/branch id to tables and reservations, and run the service on a server with a web or mobile front end.

Q82. What would you improve with more time?

Joining tables for large groups, SMS paging for waitlisted guests, a configurable dining window and a visual floor map.

Tips for answering

- Answer in two sentences, then stop. If they want more, they will ask.
- Point at the code or the running app whenever you can - "this method here..." is more convincing than theory.
- If you do not know, say how you would find out: "I would check the javadoc for `TreeMap.subMap`" is better than guessing.
- Use the project's own numbers: 90-minute window, 30-minute hold, 2-second reset, 15-second sweep.